11.2 Join table

The join table inheritance resembles the Domain Model class diagram since each class is mapped to an individual table. The subclass tables have a foreign key column referencing the base class table primary key.

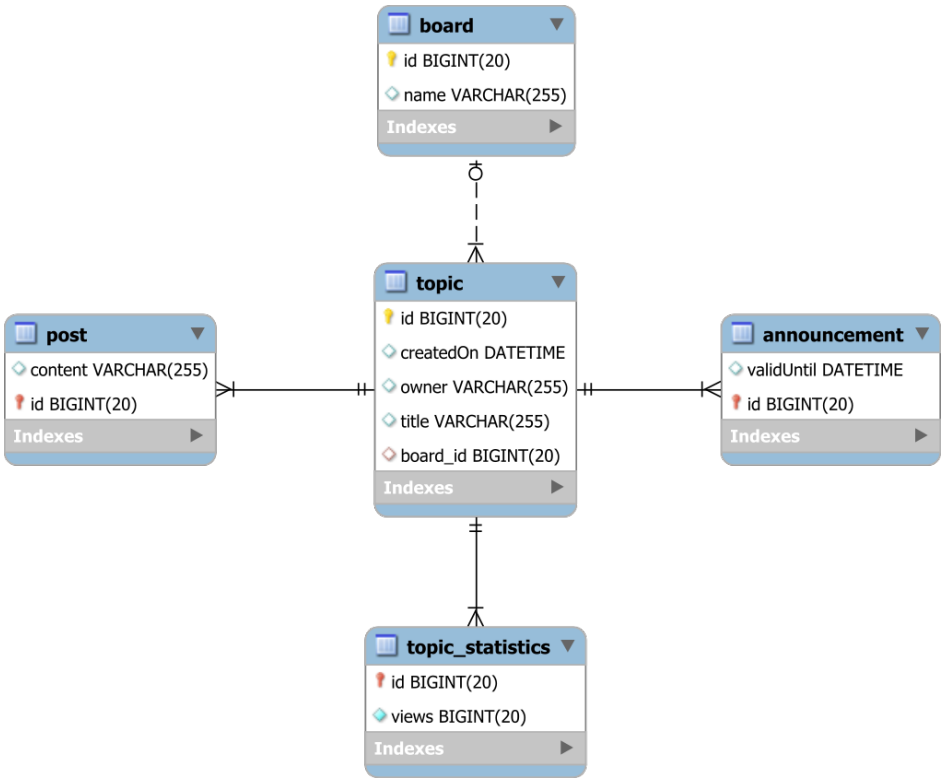

Figure 11.3: Join table

To use this inheritance strategy, the `Topic` entity must be annotated with:

```
@Inheritance(strategy = InheritanceType.JOINED)
```

The `Post` and the `Announcement` entities can use a `@PrimaryKeyJoinColumn` mapping to define the base class foreign key column.

By default, the subclass table primary key column is used as a foreign key as well.

When persisting the same entities defined in the single table section, Hibernate generates the following SQL statements:

```
INSERT INTO topic (board_id, createdOn, owner, title, id)
VALUES (1, '2016-01-17 09:27:10.694', 'John Doe', 'Inheritance', 2)

INSERT INTO post (content, id) VALUES ('Best practices', 2)

INSERT INTO topic (board_id, createdOn, owner, title, id)
VALUES (1, '2016-01-17 09:27:10.694', 'John Doe', 'Release x.y.z.Final', 3)

INSERT INTO announcement (validUntil, id) VALUES ('2016-02-17 09:27:10.698', 3)

INSERT INTO topic_statistics (views, id) VALUES (1, 2)

INSERT INTO topic_statistics (views, id) VALUES (1, 3)
```

The base class information goes into the `topic` table while the subclass content goes in the `post` or the `announcement` tables. When fetching all `Topic` entities associated with a specific `Board`:

```
List<Topic> topics = entityManager.createQuery(
    "select t from Topic t where t.board.id = :boardId", Topic.class)
.setParameter("boardId", 1L)
.getResultList();
```

Hibernate must join the base class with each individual subclass table.

```
SELECT
    t.id AS id1_3_,
    t.board_id AS board_id5_3_,
    t.createdOn AS createdO2_3_,
    t.owner AS owner3_3_,
    t.title AS title4_3_,
    t1_.content AS content1_2_,
    t2_.validUntil AS validUnt1_0_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
        WHEN t2_.id IS NOT NULL THEN 2
        WHEN t.id IS NOT NULL THEN 0
    END AS clazz_
FROM topic t
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE t.board_id = 1
```

When loading a `TopicStatistics` entity along with its `Topic` association:

```
TopicStatistics statistics = entityManager.createQuery(
    "select s from TopicStatistics s join fetch s.topic t where t.id = :topicId"
    , TopicStatistics.class)
.setParameter("topicId", topicId)
.getSingleResult();
```

Hibernate must join four tables to construct the desired result set:

```
SELECT
    ts.id AS id1_4_0_,
    t.id AS id1_3_1_,
    ts.views AS views2_4_0_,
    t.board_id AS board_id5_3_1_,
    t.createdOn AS createdO2_3_1_,
    t.owner AS owner3_3_1_,
    t.title AS title4_3_1_,
    t1_.content AS content1_2_1_,
    t2_.validUntil AS validUnt1_0_1_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
        WHEN t2_.id IS NOT NULL THEN 2
        WHEN t.id IS NOT NULL THEN 0
    END AS clazz_1_
FROM topic_statistics ts
INNER JOIN topic t ON ts.id = t.id
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE t.id = 2
```

Considering that the `Board` entity defines a `@OneToMany` `Topic` association:

```
@OneToMany(mappedBy = "board")
private List<Topic> topics = new ArrayList<>();
```

Fetching the collection lazily generates a separate select statement, identical to the previous `Topic` entity query.

When fetching the collection eagerly:

```
Board board = entityManager.createQuery(
    "select b from Board b join fetch b.topics where b.id = :id", Board.class)
.setParameter("id", id)
.getSingleResult();
```

Hibernate requires three joins to fetch all topic-related information.

```
SELECT
    b.id AS id1_1_0_, t.id AS id1_3_1_, b.name AS name2_1_0_,
    t.board_id AS board_id5_3_1_, t.createdOn AS createdO2_3_1_,
    t.owner AS owner3_3_1_, t.title AS title4_3_1_,
    t1_.content AS content1_2_1_, t2_.validUntil AS validUnt1_0_1_,
    CASE WHEN t1_.id IS NOT NULL THEN 1
         WHEN t2_.id IS NOT NULL THEN 2
         WHEN t.id IS NOT NULL THEN 0
    END AS clazz_1_,
    t.board_id AS board_id5_3_0__, t.id AS id1_3_0__
FROM board b
INNER JOIN topic t ON b.id = t.board_id
LEFT OUTER JOIN post t1_ ON t.id = t1_.id
LEFT OUTER JOIN announcement t2_ ON t.id = t2_.id
WHERE b.id = 1
```

Performance considerations

Unlike single table inheritance, the joined table strategy allows nullable subclass attribute columns.

When writing data, Hibernate requires two insert statements for each subclass entity, so there's a performance impact compared to single table inheritance. The index memory footprint also increases because instead of a single table primary key, the database must index the base class and all subclasses primary keys.

When reading data, polymorphic queries require joining the base class with all subclass tables, so, if there are `n` subclasses, Hibernate will need N+1 joins. The more joins, the more difficult it is for the database to calculate the most efficient execution plan.

11.3 Table-per-class

The table-per-class inheritance model has a table layout similar to the joined table strategy, but, instead of storing base class columns in the `topic` table, each subclass table also stores columns from the `topic` table. There is no foreign key between the `topic` and the `post` or `announcement` subclass tables, and there is no foreign key in the `topic_statistics` table either.

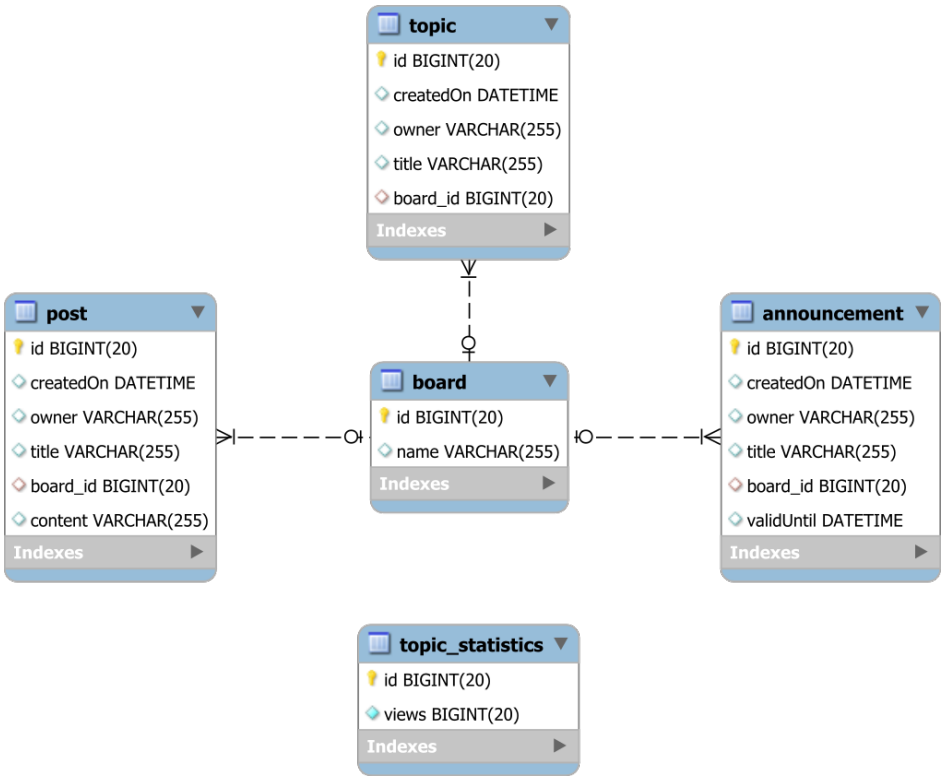

Figure 11.4: Table-per-class

To use this inheritance strategy, the `Topic` must be annotated with `@Inheritance(strategy = InheritanceType.TABLE_PER_CLASS)`.

Inserting the `Post` and the `Announcement` entities defined in the single table inheritance section generates the following SQL statements:

```
INSERT INTO post (board_id, createdOn, owner, title, content, id)
VALUES (1, '2016-01-17 09:31:12.018', 'John Doe', 'Inheritance',
        'Best practices', 2)

INSERT INTO announcement (board_id, createdOn, owner, title, validUntil, id)
VALUES (1, '2016-01-17 09:31:12.018', 'John Doe', 'Release x.y.z.Final',
        '2016-02-17 09:31:12.023', 3)
```